

BMS Dashboard Builder — Take-Home Technical Test

Overview

You are tasked with building a **Building Management System (BMS) Dashboard** — a full-stack web application that visualizes building performance data through a configurable, drag-and-drop dashboard builder.

The application must allow users to compose a personalized dashboard by selecting, placing, and configuring visualization cards. Data is stored in a **SQL Server** database, processed by a **Node.js backend**, and rendered on a **Next.js frontend**. Each card dynamically binds to data axes chosen by the user at configuration time.

Time limit: 2 days

Scope: Full-stack application (Next.js frontend + Node.js backend + SQL Server)

Data: Pre-generated CSV files are provided in the `data/` directory for import into SQL Server

Business Context

You are building a BMS dashboard for a property management company overseeing two commercial office buildings (**BLD-001** and **BLD-002**). The facility operations team needs a single-pane view into energy consumption, HVAC performance, space occupancy, and system alerts across both buildings, multiple floors, and individual zones.

The key differentiator of this dashboard is its **builder-first design**: instead of a fixed layout, the operations team can compose their own dashboards by dragging cards onto a canvas and configuring each card's data source, axes, and filters.

Data Sources

Four CSV files are provided in the `data/` directory:

File	Description	Approx. Records
<code>energy_consumption.csv</code>	Hourly energy readings per device	~80 rows
<code>hvac_performance.csv</code>	Hourly HVAC unit telemetry	~35 rows
<code>occupancy.csv</code>	Hourly occupancy and air quality per zone	~63 rows
<code>alerts_events.csv</code>	System alerts, alarms, and events	~20 rows

All files use standard CSV format with headers in the first row. Timestamps are in ISO 8601 format (`YYYY-MM-DD HH:MM:SS`). These CSV files serve as the initial data seed — candidates must import them into SQL Server and build the backend API to serve processed data to the frontend.

Refer to `data/DATA_DICTIONARY.md` for complete field definitions.

Requirements

1. Dashboard Builder (Core Feature)

Build a **drag-and-drop dashboard canvas** that allows users to:

- **Add cards** to the dashboard by dragging from a card palette/toolbar
- **Rearrange cards** on the canvas via drag-and-drop
- **Remove cards** from the dashboard
- **Resize cards** (optional, bonus)
- **Persist layout** across page refreshes (localStorage is acceptable)

The canvas should support a grid or free-form layout. A minimum of **4 cards visible simultaneously** should be supported without overflow issues.

2. Card Types

Four card types must be implemented:

Card Type	Purpose	Required Configuration
KPI Card	Displays a single aggregated metric (sum, avg, min, max, count)	Data source, metric field, aggregation type, optional filter
Bar Chart	Compares categorical data with bars	Data source, X-axis (category), Y-axis (value), aggregation, optional filter
Line Chart	Shows trends over time	Data source, X-axis (timestamp), Y-axis (value), aggregation, optional group-by
Gauge Chart	Shows a value within a min-max range	Data source, metric field, aggregation, min value, max value, target value

3. Dynamic Data Axis Selection

Each card must allow the user to **dynamically select data axes** from the available database columns:

- When a card is added to the canvas, it enters a **configuration mode**
- The user selects a **data source** (one of the 4 database tables)
- The frontend requests available columns from the backend API for the selected source
- Based on the response, the available columns are populated as options
- The user maps columns to card-specific axes:
 - **X-axis / Category** — typically a categorical or timestamp column
 - **Y-axis / Value** — typically a numeric column
 - **Group-by** (Line Chart) — an optional series column
 - **Filter** — optional column-value filter
- The frontend sends the configuration to the backend, which constructs and executes the appropriate SQL query
- The card renders immediately with the returned data
- The user can **reconfigure** a card at any time by clicking an edit/settings button

4. Filtering

The dashboard should support **global filters** applied across all cards:

- **Building** — filter by `building_id` (BLD-001, BLD-002)
- **Floor** — filter by floor number

- **Time Range** — filter by timestamp range (at minimum: today, last 7 days, custom)

Global filters update all cards on the canvas simultaneously by re-querying the backend.

5. Floor Plan View

Build a dedicated **Floor Plan page** that visualizes real-time occupancy across building zones using an SVG-based floor plan.

- Create a **simplified SVG floor plan** for each building/floor combination. This is an **imaginative floor plan** — use your own creativity to draw the layout. There is no reference floor plan provided. Represent rooms/zones as labeled polygons or rectangles arranged in a logical office layout (e.g., open work areas, meeting rooms, server room, reception). The layout does not need to match a real building — just ensure each zone from the CSV data (Zone-A , Zone-B , Zone-C) is represented as a distinct region on the SVG.
- Each **zone** on the floor plan should be an interactive overlay that:
 - Shows a **color-coded fill** based on occupancy rate (e.g., green = low, yellow = medium, red = high)
 - Displays the **zone name** and **current person count** as a label inside the zone
- On **hover/tap**, a **tooltip** appears showing:
 - Zone name and floor
 - Latest occupancy rate (%)
 - Person count / zone capacity
 - Latest CO₂ reading (ppm)
 - Latest air quality index
 - Timestamp of the reading
- The floor plan should support **building and floor selection** (dropdown or tabs) to switch between BLD-001 Floor 1, BLD-001 Floor 2, BLD-002 Floor 1, BLD-002 Floor 2
- Zone overlay data is fetched from the backend API (GET /api/occupancy/latest? building_id=...&floor=...)
- The floor plan should **auto-refresh** every 30 seconds to reflect the latest data
- Zones with **no recent data** (stale > 1 hour) should show a gray overlay with a "No data" indicator

This page is accessible from the main navigation alongside the Dashboard Builder.

6. Visual Design

- Clean, professional UI suitable for a facilities management context
- Responsive layout (must work at 1280px minimum width)
- Color scheme should differentiate alert severity levels (Critical = red, Warning = orange, Info = blue)
- Cards should display a clear title, the configured metric, and visual representation
- Loading states should be shown while data is being processed
- Empty states should guide the user to configure unconfigured cards

Technical Constraints

Frontend

- **Framework:** Next.js (App Router) — React-based with server-side rendering and API routes
- **Charts:** Any charting library compatible with React (Recharts, Chart.js, D3.js, ECharts, ApexCharts, Victory, etc.)
- **Drag & Drop:** Any DnD library (dnd-kit, react-beautiful-dnd, SortableJS, etc.)

- **Styling:** Tailwind CSS, CSS Modules, or styled-components

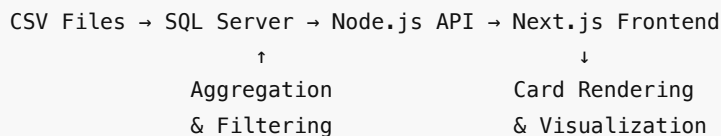
Backend

- **Runtime:** Node.js (integrated via Next.js API Routes / Route Handlers)
- **Language:** TypeScript preferred, JavaScript acceptable
- **API Design:** RESTful endpoints or Next.js Server Actions

Database

- **RDBMS:** Microsoft SQL Server (MSSQL)
- **ORM:** Prisma — candidates must define the schema in `schema.prisma` and use Prisma Client for all database queries
- **Schema:** Candidates must design the Prisma schema from the CSV data and run `prisma db push` or `prisma migrate dev` to create the SQL Server tables
- **Seeding:** Use Prisma's built-in seeding mechanism (`prisma db seed`) or a custom seed script to import CSV data into SQL Server

Data Flow



- **No client-side CSV parsing** — all data must come from the backend API
- The backend is responsible for querying, aggregating, and filtering data
- The frontend sends card configuration (data source, axes, filters) to the backend
- The backend constructs queries using Prisma Client and returns JSON responses

Deliverables

1. **Source Code** — Complete, runnable full-stack application
 2. **README** — Setup and run instructions (in addition to this file)
 3. **Architecture Brief** — A short document (or section in README) explaining:
 - State management approach
 - Data flow from SQL Server → card rendering
 - Drag-and-drop implementation strategy
 - How dynamic axis binding works
 - Database schema design decisions
 - SVG floor plan zone overlay approach
 4. **AI Prompt History** — A markdown file (`PROMPT_HISTORY.md`) documenting your complete interaction with any AI tools used during this test. This helps us understand your prompting strategy and problem-solving approach. Include:
 - All prompts/questions you sent to the AI
 - The AI's responses (summarized or full, at your discretion)
 - Any follow-up refinements or iterations
 - How you used AI-generated suggestions vs. your own decisions
 5. **Database Schema & Seed Script** — Prisma schema file (`schema.prisma`) and seed script that define the database structure and import CSV data into SQL Server. Include the generated Prisma Client in your source code.
-

Evaluation Criteria

Category	Weight	What We're Looking For
Functionality	30%	All 4 card types work correctly; drag-and-drop is functional; dynamic axis selection works; filters apply correctly; floor plan page renders zones with occupancy overlays and tooltips
Code Quality	20%	Clean component structure; separation of concerns; meaningful naming; no dead code; consistent patterns
Data Handling	20%	Prisma schema design is sound; CSV import works correctly; aggregation logic is correct; Prisma queries are used properly (no raw SQL injection risks); filtering works across cards; handles edge cases (empty data, missing values)
UI/UX	15%	Professional appearance; intuitive builder flow; proper loading/empty states; responsive layout
Architecture	10%	Sound state management; clear frontend/backend separation; extensible design for adding new card types; efficient SQL queries; proper error handling across the stack
Prompt History	5%	Completeness of PROMPT_HISTORY.md; clarity of prompting strategy; evidence of iterative refinement; quality of AI-assisted problem solving

Note on AI Tool Usage: Using AI assistants (ChatGPT, Copilot, Claude, etc.) is permitted and encouraged. The `PROMPT_HISTORY.md` file is a scored deliverable — it demonstrates your ability to effectively leverage AI tools in your workflow.

Bonus Points (Optional)

These are not required but will be appreciated:

- ☐ **Card resizing** — Allow cards to be resized on the canvas (e.g., 1×1, 2×1, 2×2)
- ☐ **Export/Import layout** — Save dashboard configuration as JSON and reload it
- ☐ **Dark mode toggle**
- ☐ **Animated transitions** — Smooth animations when adding/removing/rearranging cards
- ☐ **Card duplication** — Clone an existing card with its configuration
- ☐ **Real-time clock** — Show current time and highlight the "current" data point on line charts
- ☐ **Print/PDF export** — Render the dashboard for printing
- ☐ **Unit tests** — Test aggregation functions, query builder, and filter logic
- ☐ **Query logging** — Log all executed SQL queries with execution time for debugging